

Laboratório de Concorrência · Aula 10

Gustavo de Almeida Gomes · RA 24000271-2 · Turma A

Gerado em 09/10/2026, 20:33:40 · tempo registrado no cronômetro: 0:00

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

A sincronização de cooperação acontece quando uma pessoa precisa esperar a outra terminar alguma coisa. Por exemplo, esperar alguém terminar de usar o banheiro para poder entrar. Já a sincronização de competição acontece quando duas pessoas querem usar o mesmo recurso ao mesmo tempo, como duas pessoas querendo usar o mesmo computador, por exemplo.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

Espero que o programa mostre 2.000.000, porque cada thread vai somar 1 ao contador um milhão de vezes.

1ª execução (n = 1.000.000)

1014840

2ª execução (n = 1.000.000)

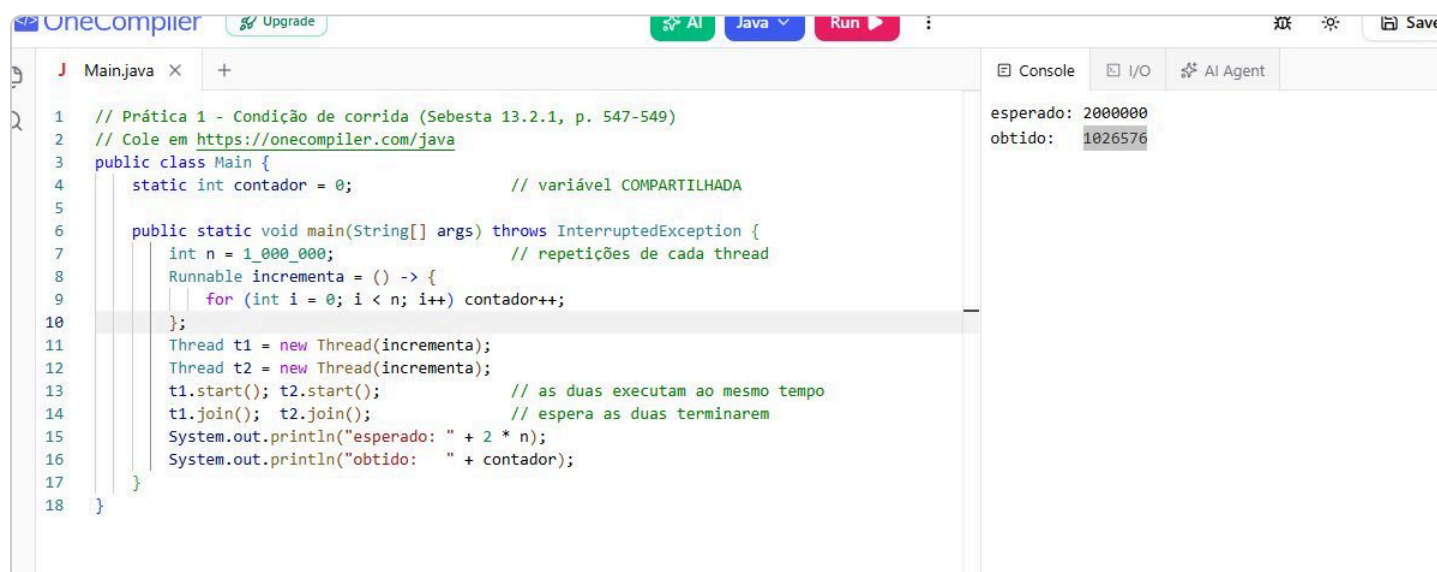
1357727

3ª execução (n = 1.000.000)

1026576

Saída do programa

não respondido



```
1 // Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
2 // Cole em https://onecompiler.com/java
3 public class Main {
4     static int contador = 0; // variável COMPARTILHADA
5
6     public static void main(String[] args) throws InterruptedException {
7         int n = 1_000_000; // repetições de cada thread
8         Runnable incrementa = () -> {
9             for (int i = 0; i < n; i++) contador++;
10        };
11        Thread t1 = new Thread(incrementa);
12        Thread t2 = new Thread(incrementa);
13        t1.start(); t2.start(); // as duas executam ao mesmo tempo
14        t1.join(); t2.join(); // espera as duas terminarem
15        System.out.println("esperado: " + 2 * n);
16        System.out.println("obtido: " + contador);
17    }
18 }
```

Console

esperado: 2000000
obtido: 1026576

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não, e foram diferentes nas três execuções. Isso acontece porque as duas threads acessam a mesma variável ao mesmo tempo, fazendo com que algumas atualizações sejam perdidas. Como a ordem de execução muda, o resultado também pode mudar a cada execução, como foi o caso.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

Isso acontece quando as duas threads leem o mesmo valor do contador antes de gravar. Por exemplo, se o contador vale 10, a thread 1 lê 10 e soma 1, chegando a 11. Antes de gravar, a thread 2 também lê 10 e soma 1. As duas gravam 11, então uma atualização se perde e o contador fica com 11 em vez de 12.

1.3 Com $n = 1000$, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com $n = 1000$, o resultado pode ter sido o esperado, porque, com menos repetições, fica mais difícil acontecer uma condição de corrida. Mas ao mesmo tempo, também pode ser que ocorram resultados diferentes também.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8

BBBAAA → 7

BABABA → 4

AABBAB → 6

Saída do programa

não respondido

```

8 def executa(ordem):
    # ordem: por exemplo "AABABB"
11     passo = {"A": 0, "B": 0}
12     for t in ordem:
13         if passo[t] == 0:
14             reg[t] = total
15             # 1) ler
16         elif passo[t] == 1:
17             reg[t] = reg[t] + 1 if t == "A" else reg[t] * 2
18             # 2) calcular
19         else:
20             total = reg[t]
21             # 3) gravar
22             passo[t] += 1
23     return total
24
25 print("AAABBB ->", executa("AAABBB")) # A inteira, depois B inteira
26 print("BBBAAA ->", executa("BBBAAA")) # B inteira, depois A inteira
27 print("BABABA ->", executa("BABABA")) # alternando passo a passo
28
29 resultados = {}
30 for posicao in combinations(range(6), 3):
    # onde ficam os 3 passos de A

```

Console

```

AAABBB -> 8
BBBAAA -> 7
BABABA -> 6
Ordens possíveis: 20
TOTAL = 4: 9 ordens, por exemplo AABBBB
TOTAL = 6: 9 ordens, por exemplo AABABB
TOTAL = 7: 1 ordens, por exemplo BBBAAA
TOTAL = 8: 1 ordens, por exemplo AAABBB

```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

Existem 20 ordens possíveis de execução e 4 resultados diferentes: 4, 5, 7 e 8. Dessas ordens, 6 produzem o resultado correto, que é 8.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Uma ordem que dá 4 é BAAABB. Primeiro, B lê o valor 3 e calcula 6, mas ainda não grava. Depois, A lê 3 e calcula 4. B grava 6, mas A grava 4 no final, trocando o valor de B. Assim, o resultado final é 4, porque a atualização de B foi sobrescrita pela de A.

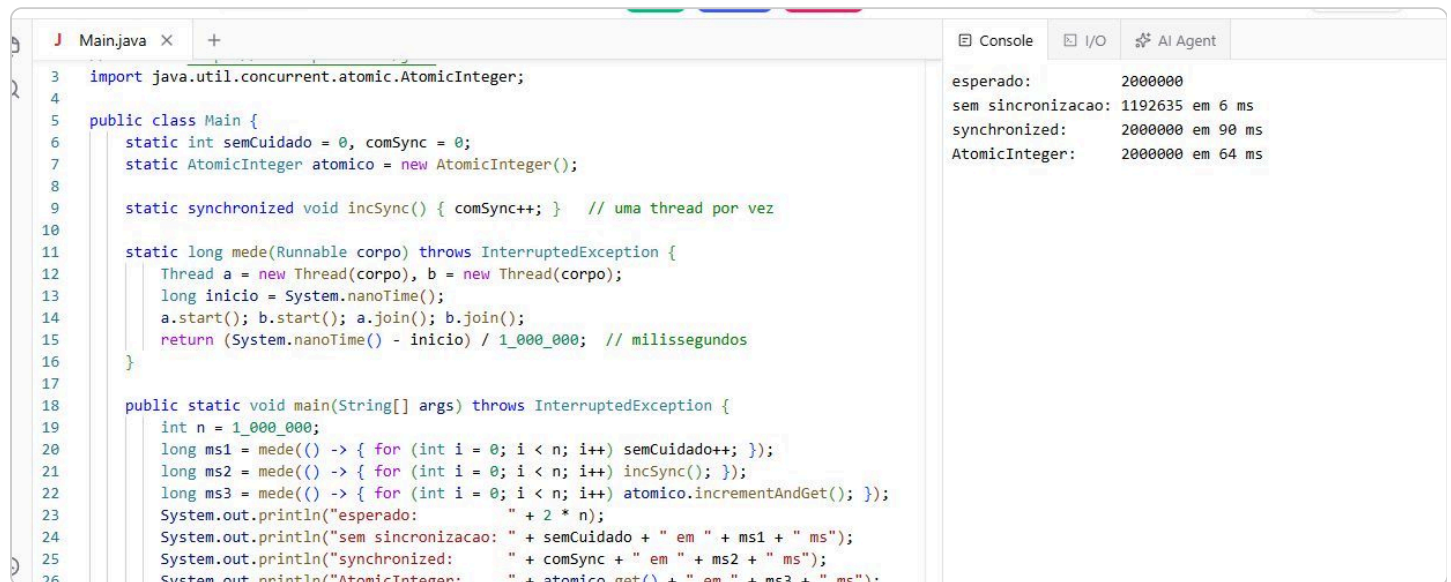
Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Acredito que a versão sem sincronização será a mais rápida, porque não precisa controlar o acesso à variável. Porém, ela pode apresentar um resultado incorreto. Já as versões com `synchronized` e `AtomicInteger` devem apresentar o resultado correto, pois evitam a perda de atualizações.

Saída do programa

não respondido



The screenshot shows a Java IDE with a file named `Main.java`. The code implements a concurrent program with three different synchronization methods: no synchronization, `synchronized`, and `AtomicInteger`. The `main` method sets `n = 1_000_000` and measures the execution time for each method. The console output on the right shows the results:

Method	Result	Time
esperado:	2000000	
sem sincronizacao:	1192635	em 6 ms
synchronized:	2000000	em 90 ms
AtomicInteger:	2000000	em 64 ms

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

As versões com `synchronized` e `AtomicInteger` deram o valor correto de 2.000.000, enquanto a versão sem sincronização pode apresentar um valor menor. As versões corretas costumam ser mais lentas porque precisam controlar o acesso à variável compartilhada para evitar que as threads percam atualizações.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um `AtomicInteger` em cada saldo resolve? Por quê? O que seria preciso?

Não, porque cada `AtomicInteger` garante a atualização de um saldo individualmente, mas não garante que a transferência inteira aconteça corretamente. Seria necessário sincronizar as duas operações para garantir que o dinheiro seja retirado de uma conta e colocado na outra sem interferência de outras transações.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Não, nunca irão aparecer. A função `pausa()` garante que cada thread pegue um recurso e demore 100 ms para tentar pegar o segundo, sendo assim criado um deadlock.

Saídas (antes e depois da correção)

não respondido

```

1 // Prática 4 - Impasse (deadlock) (Sebesta 13.2.1, p. 551)
2 // Cole em https://onecompiler.com/java
3 import java.lang.management.ManagementFactory;
4
5 public class Main {
6     static final Object X = new Object(), Y = new Object(); // dois recursos
7
8     static void pausa() {
9         try { Thread.sleep(100); } catch (InterruptedException e) { }
10    }
11
12    public static void main(String[] args) throws Exception {
13        Thread a = new Thread() -> {
14            synchronized (X) { pausa(); synchronized (Y) { System.out.println("A terminou"); } }
15        };
16        Thread b = new Thread() -> {
17            synchronized (Y) { pausa(); synchronized (X) { System.out.println("B terminou"); } }
18        };
19        a.start(); b.start();
20        Thread.sleep(1000); // espera 1 segundo
21        long[] presas = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
22        System.out.println(presas == null ? "sem impasse"
23            : "IMPASSE: " + presas.length + " threads presas");
24        System.exit(0); // encerra o programa
25    }
26 }

```

IMPASSE: 2 threads presas

```

1 // Prática 4 - Impasse (deadlock) (Sebesta 13.2.1, p. 551)
2 // Cole em https://onecompiler.com/java
3 import java.lang.management.ManagementFactory;
4
5 public class Main {
6     static final Object X = new Object(), Y = new Object(); // dois recursos
7
8     static void pausa() {
9         try { Thread.sleep(100); } catch (InterruptedException e) { }
10    }
11
12    public static void main(String[] args) throws Exception {
13        Thread a = new Thread() -> {
14            synchronized (X) { pausa(); synchronized (Y) { System.out.println("A terminou"); } }
15        };
16        Thread b = new Thread() -> {
17            synchronized (X) { pausa(); synchronized (Y) { System.out.println("B terminou"); } }
18        };
19        a.start(); b.start();
20        Thread.sleep(1000); // espera 1 segundo
21        long[] presas = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
22        System.out.println(presas == null ? "sem impasse"
23            : "IMPASSE: " + presas.length + " threads presas");
24        System.exit(0); // encerra o programa
25    }
26 }

```

A terminou
B terminou
sem impasse

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

Thread A pega para utilizar o recurso X ao mesmo tempo que a Thread B pega para utilizar o recurso Y. Depois, A tenta pegar Y, mas não consegue, pois está sendo executado por B então, a thread A espera B finalizar o uso do recurso Y. Ao mesmo tempo, B tenta pegar X, mas não consegue, por conta de estar sendo utilizado por X então, fica esperando que A libere o recurso X.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Com as duas threads solicitando os mesmos recursos na mesma ordem, a espera circular nunca iria acontecer. Sendo assim, A thread A iria pegar o recurso X, então a thread B teria que esperar A terminar para começar a executar. Então, a thread A iria usar X e depois Y e liberar os dois recursos para que B começasse a sua execução.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

No total levaria 200 ms para que 3 trabalhadores finalizem as 6 tarefas.

Com 1 trabalhador levaria 600ms

com 600 trabalhadores levaria 100ms

Tempo com 1 trabalhador

600

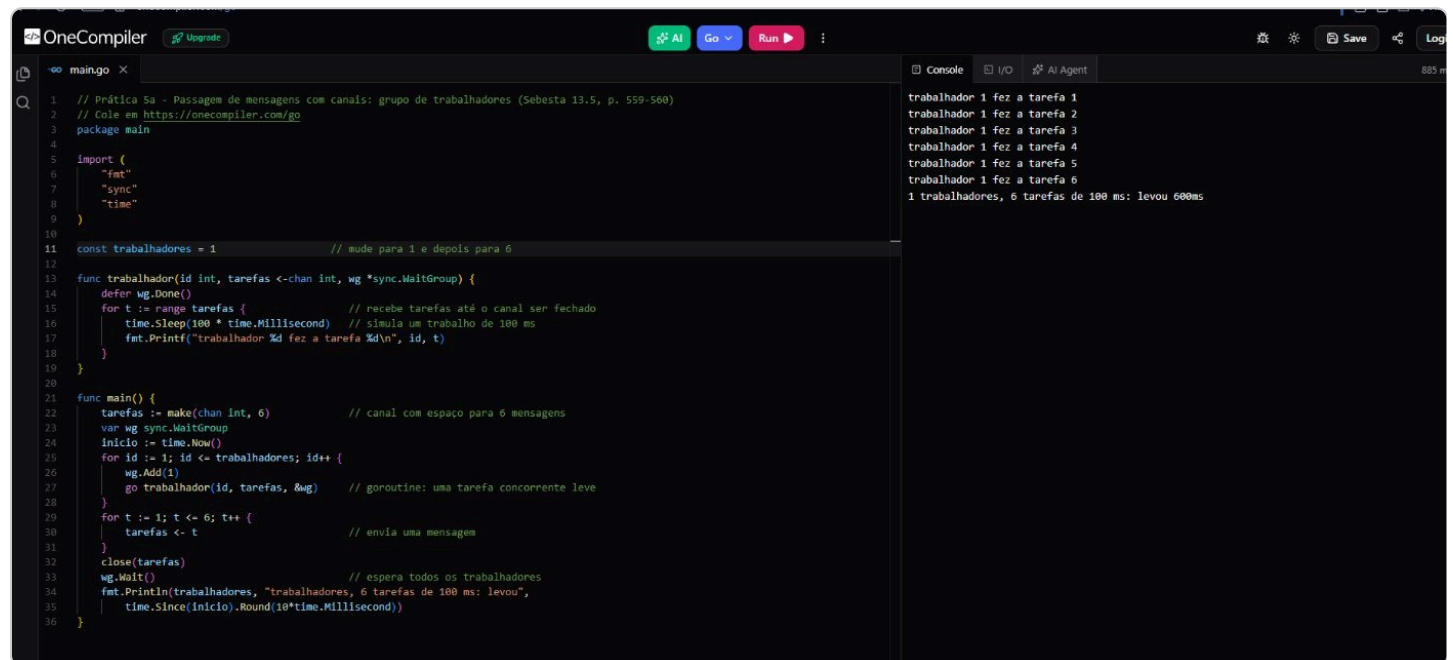
Tempo com 3 trabalhadores

200

Tempo com 6 trabalhadores

100

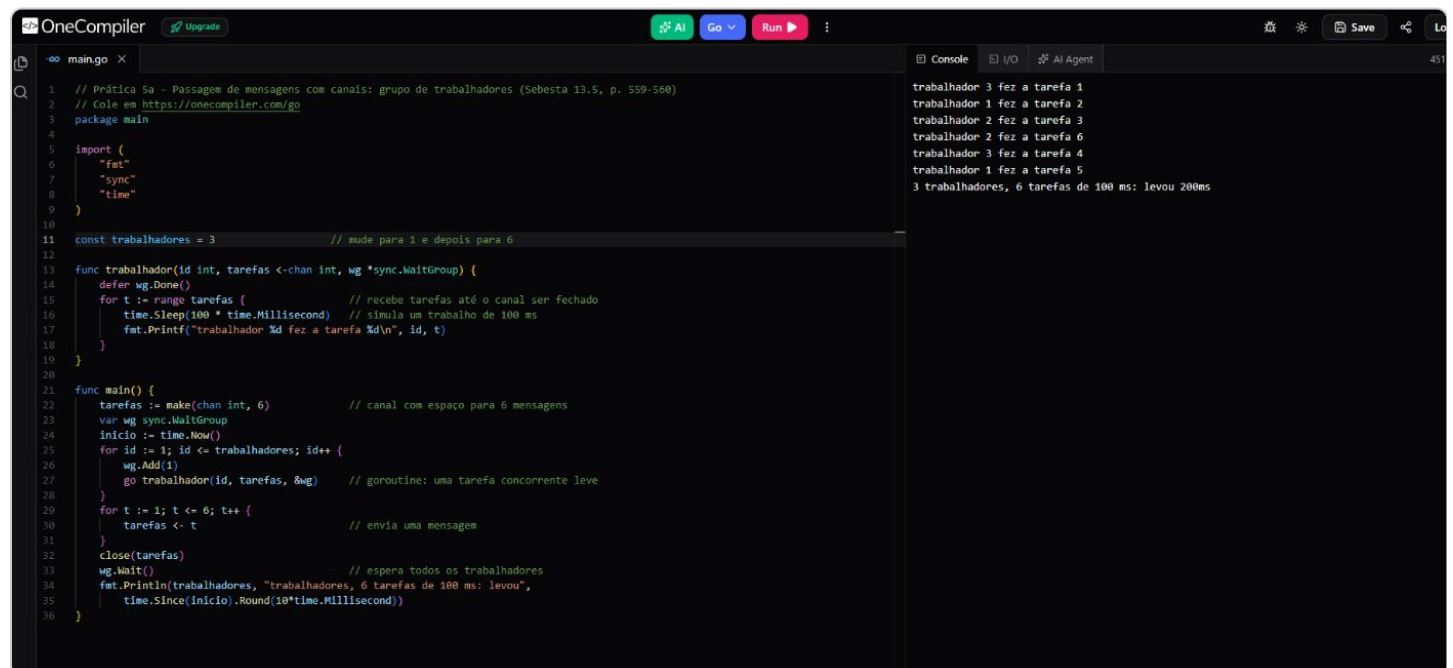
Saída do programa

não respondido

The screenshot shows the OneCompiler interface with a Go program. The code defines a worker function that processes 6 tasks, each taking 100ms. In the main function, a channel is created with a capacity of 6, and a single worker is launched. The main function then waits for the worker to finish and prints the total time. The console output shows the worker completing all 6 tasks and the final time being 600ms.

```
1 // Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p. 559-560)
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import (
6     "fmt"
7     "sync"
8     "time"
9 )
10
11 const trabalhadores = 1 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
23     var wg sync.WaitGroup
24     inicio := time.Now()
25     for id := 1; id <= trabalhadores; id++ {
26         wg.Add(1)
27         go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
28     }
29     for t := 1; t <= 6; t++ {
30         tarefas <- t // envia uma mensagem
31     }
32     close(tarefas)
33     wg.Wait() // espera todos os trabalhadores
34     fmt.Println(trabalhadores, "trabalhadores, 6 tarefas de 100 ms: levou",
35         time.Since(inicio).Round(10*time.Millisecond))
36 }
```

trabalhador 1 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 1 fez a tarefa 3
trabalhador 1 fez a tarefa 4
trabalhador 1 fez a tarefa 5
trabalhador 1 fez a tarefa 6
1 trabalhadores, 6 tarefas de 100 ms: levou 600ms



The screenshot shows the same Go program but with the number of workers set to 3. The console output shows the tasks being distributed among 3 workers, and the final time being 200ms.

```
1 // Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p. 559-560)
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import (
6     "fmt"
7     "sync"
8     "time"
9 )
10
11 const trabalhadores = 3 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
23     var wg sync.WaitGroup
24     inicio := time.Now()
25     for id := 1; id <= trabalhadores; id++ {
26         wg.Add(1)
27         go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
28     }
29     for t := 1; t <= 6; t++ {
30         tarefas <- t // envia uma mensagem
31     }
32     close(tarefas)
33     wg.Wait() // espera todos os trabalhadores
34     fmt.Println(trabalhadores, "trabalhadores, 6 tarefas de 100 ms: levou",
35         time.Since(inicio).Round(10*time.Millisecond))
36 }
```

trabalhador 3 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 2 fez a tarefa 3
trabalhador 2 fez a tarefa 6
trabalhador 3 fez a tarefa 4
trabalhador 1 fez a tarefa 5
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms

```

1 // Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p. 559-560)
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import (
6     "fmt"
7     "sync"
8     "time"
9 )
10
11 const trabalhadores = 6 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
23     var wg sync.WaitGroup
24     inicio := time.Now()
25     for id := 1; id <= trabalhadores; id++ {
26         wg.Add(1)
27         go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
28     }
29     for t := 1; t <= 6; t++ {
30         tarefas <- t // envia uma mensagem
31     }
32     close(tarefas)
33     wg.Wait() // espera todos os trabalhadores
34     fmt.Println(trabalhadores, "tarefas de 100 ms: levou",
35         time.Since(inicio).Round(10*time.Millisecond))
36 }

```

Console output:

```

trabalhador 2 fez a tarefa 3
trabalhador 6 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 3 fez a tarefa 4
trabalhador 4 fez a tarefa 5
trabalhador 5 fez a tarefa 6
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms

```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Pois, nesse sistema, cada trabalhador seria independente do outro trabalhador. Sendo assim, quando um trabalhador finaliza uma tarefa, ele já pegaria a próxima disponível, não sendo necessário aguardar a comunicação com outro trabalhador. Não precisa de travas porque nada é compartilhado e modificado pelas rotinas, cada tarefa é uma mensagem para que a comunicação ocorra

Saídas (antes e depois da troca)

não respondido

```

1 // Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p. 559-560)
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import (
6     "fmt"
7     "sync"
8     "time"
9 )
10
11 const trabalhadores = 6 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     ch := make(chan int) // canal SEM espaço (sem buffer)
23     ch <- 1 // envia... e espera alguém receber
24     fmt.Println(<-ch)
25 }

```

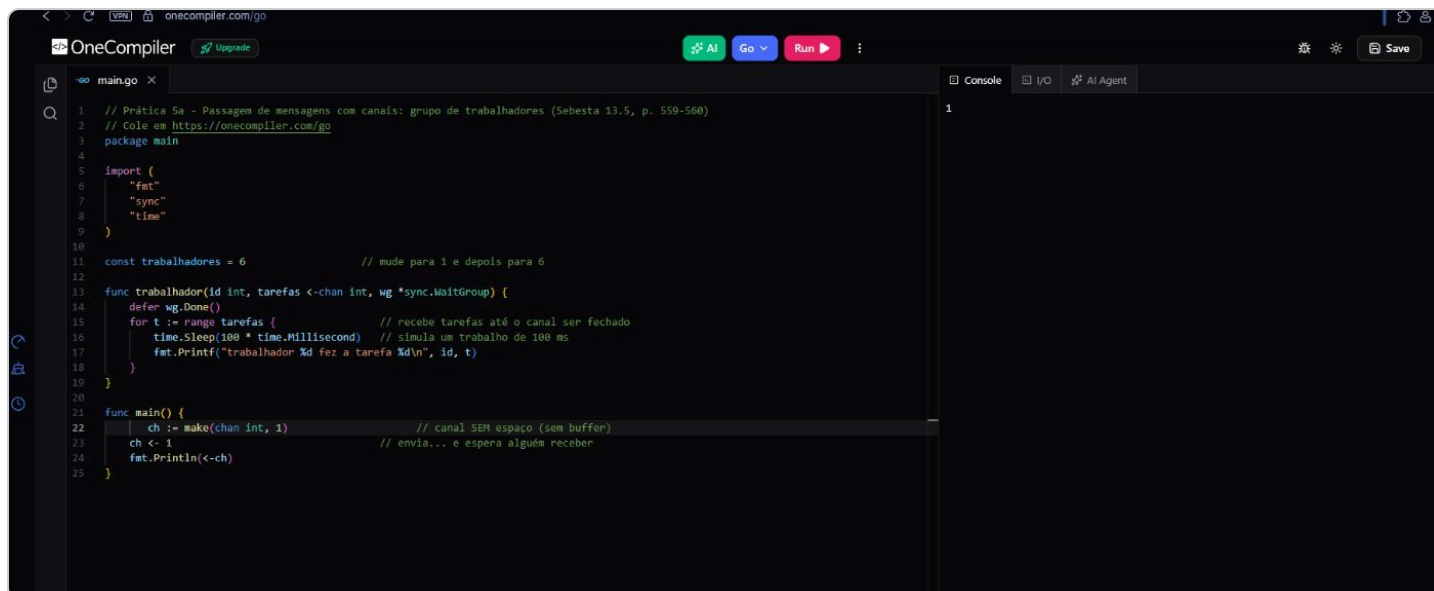
Console output:

```

fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
    /box/main.go:23 +0x38
exit status 2

```



```
1 // Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p. 559-560)
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import (
6     "fmt"
7     "sync"
8     "time"
9 )
10
11 const trabalhadores = 6 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     ch := make(chan int, 1) // canal SEM espaço (sem buffer)
23     ch <- 1 // envia... e espera alguém receber
24     fmt.Println(<-ch)
25 }
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O primeiro programa entra em deadlock, basicamente porque o canal criado com `make(chan int)` não possui capacidade de armazenamento para mensagens e consequentemente, o emissor precisa aguardar que outra goroutine esteja pronta para receber. Nesse código, existe apenas a goroutine `main` ela permanece bloqueada na linha `ch <- 1` aguardando um receptor, embora a operação `<-ch` destinada a receber a mensagem esteja logo na linha seguinte — uma linha que nunca chega a ser executada. Assim, a `main` aguarda uma ação que apenas ela mesma poderia realizar posteriormente; o Go detecta que todas as goroutines estão bloqueadas e encerra o programa, relatando um erro de deadlock. Com `make(chan int, 1)`, o canal passa a ter capacidade para uma mensagem portanto, a operação de envio simplesmente armazena o valor nesse espaço e prossegue sem esperar. Depois, `<-ch` recupera o valor do canal e o programa imprime 1

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

o compilador não deixa rodar, porque a thread pode continuar viva depois que `main` terminar e a variável contador pertence a `main`.

Saídas (o erro e o resultado com `move`)

não respondido

```

1 // Prática 6a - Rust: tentando compartilhar uma variável entre threads
2 // Cole em https://onecompiler.com/rust
3 use std::thread;
4
5 fn main() {
6     let mut contador = 0;
7     let t = thread::spawn(|| {
8         contador += 1; // a thread altera a variável de main
9     });
10    t.join().unwrap();
11    println!("{}", contador);
12 }

```

error[E0373]: closure may outlive the current function, but it borrows 'contador', which is owned by the current function
--> HelloWorld.rs:7:27

7 | let t = thread::spawn(|| {
8 | contador += 1; // a thread altera a variável de main
9 | });
note: function requires argument type to outlive 'static'
--> HelloWorld.rs:7:13

help: to force the closure to take ownership of 'contador' (and any other referenced variables), use the 'move' keyword

7 | let t = thread::spawn(move || {
8 | contador += 1; // a thread altera a variável de main
9 | });
note: requirement that the value outlives 'static' introduced here
-->

error[E0502]: cannot borrow 'contador' as immutable because it is also borrowed as mutable
--> HelloWorld.rs:11:20

11 | t.join().unwrap();
12 | println!("{}", contador);
note: mutable borrow occurs here

8 | contador += 1; // a thread altera a variável de main
9 | });
note: first borrow occurs due to use of 'contador' in closure

10 | t.join().unwrap();
11 | println!("{}", contador);
note: immutable borrow occurs here

Saída do programa

não respondido

```

1 // Prática 6b - Rust: compartilhar com Arc (contagem de referências) e Mutex (exclusão mútua)
2 // Cole em https://onecompiler.com/rust
3 use std::sync::{Arc, Mutex};
4 use std::thread;
5
6 fn main() {
7     let contador = Arc::new(Mutex::new(0)); // compartilhado e protegido
8     let mut threads = Vec::new();
9     for _ in 0..4 {
10        let c = Arc::clone(&contador);
11        threads.push(thread::spawn(move || {
12            for _ in 0..100_000 {
13                *c.lock().unwrap() += 1; // trava, soma e destrava
14            }
15        }));
16    }
17    for t in threads { t.join().unwrap(); }
18    println!("contador = {}", *contador.lock().unwrap());
19 }

```

contador = 400000

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Com move, uma thread recebe uma cópia do contador e o incrementa em 1, mas o valor original em main permanece 0. É por isso que 0 é exibido na impressão. O programa compila, mas o resultado não é o esperado, pq cada uma tem o seu próprio número.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java, a corrida só foi descoberta quando o programa rodou, porque o código compilava normalmente e o valor final saía errado, mudando de uma execução para outra, o que dificulta achar o problema. Já em Rust, o erro foi descoberto antes de executar, na compilação, pois o compilador não deixa a thread alterar uma variável sem proteção. Sendo assim, esse tipo de erro não chega até o usuário, e para compartilhar o dado o programador

precisa usar Arc junto com Mutex. A desvantagem é que o código fica mais difícil de escrever, pois é preciso entender as regras do compilador e seguir o que ele exige para o programa compilar.

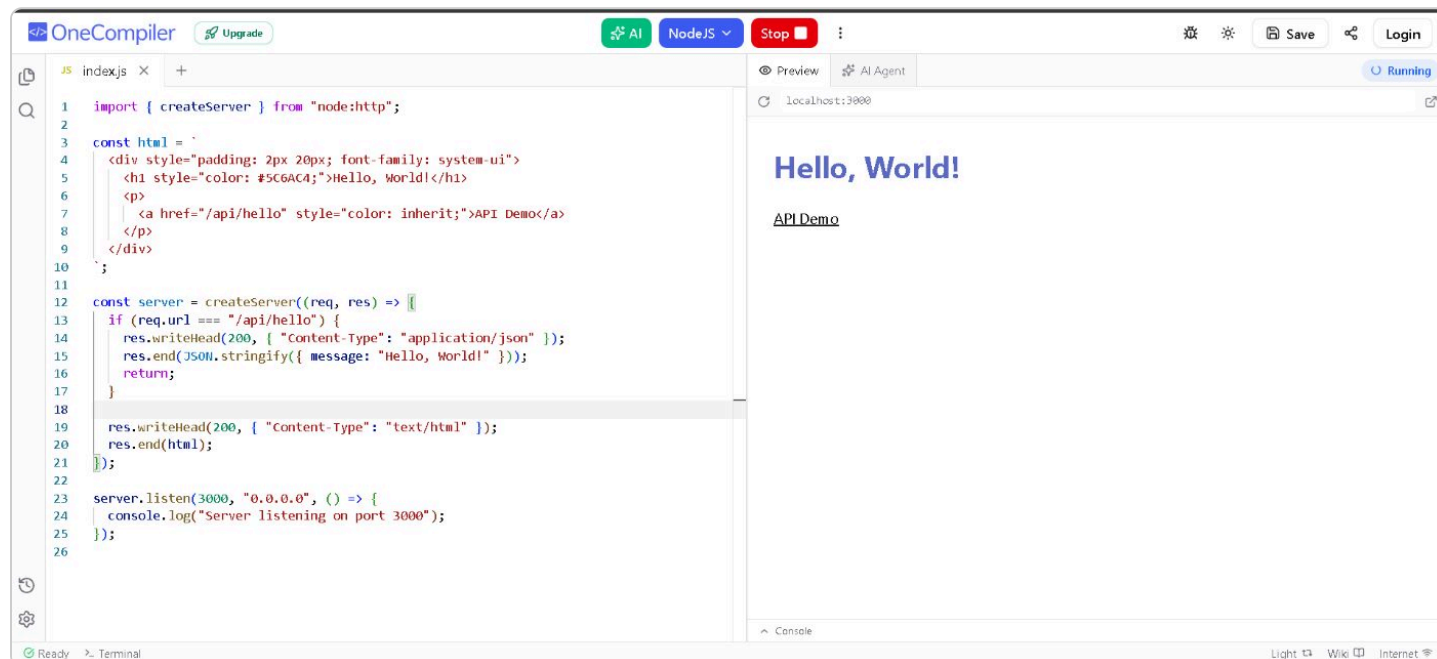
Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A, D, C, B

Saída do programa

não respondido



7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A ordem de saída é: A, D, C, B. Isto acontece porque o JavaScript executa todo o código síncrono em primeiro lugar, imprimindo as letras A e D. Em seguida, o laço de eventos processa as "microtarefas" pendentes, como é o caso das Promises, executando a função que imprime o C. Apenas no fim é que o sistema processa as tarefas agendadas ("macrotarefas"), como o setTimeout, pelo que o B é impresso em último lugar, mesmo que o seu tempo de espera tenha sido definido como 0 milissegundos.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O temporizador atrasou-se porque o JavaScript e o Node.js utilizam uma única thread com um laço de eventos (event loop). A função ocupado(300) monopolizou e bloqueou essa única thread durante cerca de 300 milissegundos, impedindo que o laço de eventos avançasse para libertar e processar o temporizador que já estava pronto. Num servidor em Node.js, isto tem um impacto drástico: um único cálculo pesado irá "prender" a thread principal, fazendo com que o servidor congele e fique incapaz de responder a todos os outros utilizadores até que o cálculo termine.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

O problema que vai ocorrer é uma clássica "condição de corrida", devido à falta de sincronização de competição. Ambas as threads podem ler o valor "1 lugar disponível" de forma simultânea (antes de qualquer uma gravar a atualização), processando ambas a venda e atribuindo o mesmo bilhete a duas pessoas diferentes. Para evitar isto, usaria mecanismos de exclusão mútua: um monitor genérico (como o `synchronized` em Java) para garantir que apenas um pedido acede ao inventário de cada vez, operações atômicas no próprio dado, ou uma arquitetura baseada na passagem de mensagens (como os canais em Go).

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

De verdade, foi a linha de aprendizado. E além disso esse Html muito bom realizado pelo professor.